

```
from google.colab import files
files.upload()
```

```
import cv2
import glob
import numpy as np
from google.colab.patches import cv2_imshow

# Chessboard settings
num_squares_x = 8
num_squares_y = 10

num_interior_corners_x = num_squares_x - 1
num_interior_corners_y = num_squares_y - 1

checker_width = 0.020

def calibrate():

    termination_criteria = (cv2.TERM_CRITERIA_EPS +
                             cv2.TERM_CRITERIA_MAX_ITER,
                             30, 0.001)

    object_points_3D = np.zeros(
        (num_interior_corners_x * num_interior_corners_y, 3),
        np.float32
    )

    object_points_3D[:, :2] = np.mgrid[
        0:num_interior_corners_y,
        0:num_interior_corners_x
    ].T.reshape(-1, 2)

    object_points_3D = object_points_3D * checker_width

    object_points = []
    image_points = []

    images = glob.glob("CAM*.PNG")

    if len(images) == 0:
        print("No images found!")
        return

    print("Images found:", images)

    img_size = None

    for image_file in images:

        img = cv2.imread(image_file)

        if img is None:
            print("Cannot read:", image_file)
            continue

        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

        img_size = gray.shape[::-1]

        success, corners = cv2.findChessboardCorners(
            gray,
            (num_interior_corners_y, num_interior_corners_x),
            None
        )

        if success:

            print("Corners detected in", image_file)

            object_points.append(object_points_3D)

            corners2 = cv2.cornerSubPix(
                gray,
```

```
        corners,
        (11,11),
        (-1,-1),
        termination_criteria
    )

    image_points.append(corners2)

    cv2.drawChessboardCorners(
        img,
        (num_interior_corners_y, num_interior_corners_x),
        corners2,
        success
    )

    cv2.imshow(img)

else:
    print("Chessboard NOT detected in", image_file)

if len(object_points) == 0:
    print("No chessboard corners detected in any image.")
    return

ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
    object_points,
    image_points,
    img_size,
    None,
    None
)

print("\nCamera Matrix:\n", mtx)
print("\nDistortion Coefficients:\n", dist)

calibrate()
```

